

Inbox X-Ray

Read Email from an Agent

Your agent reads your real Gmail inbox — for the first time.

MS 1.2

Session

MS 1.1 Done

Prereq

Gmail MCP

New Tool

Google OAuth

Auth

Windows

OS

Yesterday → Today

The same MCP loop — different server.



Yesterday (MS 1.1)

Host	Cline in VS Code
LLM	Gemini 2.0 Flash (free)
Server	Filesystem MCP server
Data	test_mcp.txt on Desktop
Proof	Agent read a local file ✓



Today (MS 1.2)

Host	Cline in VS Code
LLM	Gemini 2.0 Flash (free)
Server	Gmail MCP server NEW
Data	Your real Gmail inbox NEW
Proof	Agent lists real threads NEW

The loop doesn't change. Only the server changes.

The Plan for Today

3 steps. One payoff.

01

Google Cloud Project + OAuth Credentials

Create a project in Google Cloud Console, enable the Gmail API, and generate the OAuth client ID and secret. This is the 'key' that lets your agent talk to Gmail.

 ~15 min

02

Install & Configure the Gmail MCP Server

Clone/install the open-source Gmail MCP server, wire it to your OAuth credentials, and connect it to Cline — replacing yesterday's filesystem server.

 ~15 min

03

First Inbox Read — The Payoff

Ask Cline to list your last 10 threads. Watch the agent pull real senders, subjects, and snippets from your actual Gmail inbox. No fake data.

 ~10 min

Step 1A — Google Cloud Project

One-time setup. Takes ~5 minutes.

1

Go to `console.cloud.google.com` and sign in with your Google account

💡 *(same account as your Gmail)*

2

Click the project dropdown at the top → "New Project"

3

Name it "MCP Chief of Staff" → click Create

💡 *If you did the homework, this already exists ✓*

4

In the search bar, type `Gmail API` → click it → click Enable

💡 *Wait for the green tick — takes ~10 seconds*

✓ Check: In your Cloud Console you can see Gmail API with status Enabled

Step 1B — Create OAuth Credentials

This gives your agent permission to read Gmail.

- 1 In Cloud Console → "APIs & Services" → "Credentials"
- 2 "+ Create Credentials" → "OAuth client ID"
- 3 Application type → Desktop app → Name it MCP Gmail Agent
- 4 Click Create → Download JSON → save as credentials.json
- 5 Go to "OAuth consent screen" → Add your Gmail as a Test User

What is OAuth?

OAuth is a permission system. Instead of giving your agent your Gmail password, you give it a limited token:

- Read-only access
- To your inbox only
- Revocable any time

Your agent never sees your password.



Save credentials.json somewhere you'll remember — you'll point the MCP server at this file next.

Step 2A — Install the Gmail MCP Server

Open-source. Self-hosted. Free.

In your VS Code terminal:

Clone the server repo

```
git clone https://github.com/gongrzhe/gmail-mcp-server.git
```

Navigate into it

```
cd gmail-mcp-server
```

Install dependencies

```
npm install
```

Copy your credentials file into this folder

```
copy C:\Users\YourName\Downloads\credentials.json .
```

✓ Check: You see a `node_modules` folder inside `gmail-mcp-server` and `credentials.json` is in the same folder.

Step 2B — Wire It to Cline

Replace the filesystem server config with the Gmail server.

In Cline → MCP Servers → edit the config JSON:

```
{
  "mcpServers": {
    "gmail": {
      "command": "node",
      "args": [ "C:\\\\Users\\YourName\\gmail-mcp-server\\index.js" ],
      "env": {
        "CREDENTIALS_PATH":
"C:\\\\Users\\YourName\\gmail-mcp-server\\credentials.json"
      }
    }
  }
}
```

command: node

→ We're running a Node.js server — not npx like yesterday

Double backslashes

→ Windows JSON paths always need \\ not \

CREDENTIALS_PATH

→ Tells the server where to find your credentials.json

Step 2C — The One-Time OAuth Flow

You'll do this once. Never again.

When Cline first uses the Gmail server, a browser window opens. Here's what you'll see:

1

Google sign-in page opens in your browser

Sign in with the Google account whose inbox you want to use.

2

"MCP Gmail Agent wants to access your Gmail"

Click "Allow". You added yourself as a Test User — so this works.

3

token.json is saved automatically

The server writes a token file. Next time — no browser. Fully silent.

4

Back in Cline — the server is connected

You'll see the gmail MCP server listed as active with its tools available.

✓ Check: A file called `token.json` now exists inside your `gmail-mcp-server` folder.

What Your Agent Can Do Now

The Gmail MCP server exposes these tools automatically.

list_emails

List recent threads — returns sender, subject, snippet, thread ID, date

get_email

Fetch the full content of one email by its ID — complete body text

search_emails

Search inbox by keyword, sender, date range — same as Gmail search

send_email

Send an email — used in Sprint 3 (not today, but it's already there)

create_draft

Save a draft without sending — useful for human-in-loop review

list_labels

Return all Gmail labels — Inbox, Sent, Starred, custom labels, etc.

Today we use `list_emails` and `get_email`. The rest power Sprint 2 and Sprint 3.

Step 3 — The Payoff

Your agent reads your real inbox.

Type this into Cline →

"List my last 10 Gmail threads. Show me the sender, subject, and a short snippet for each."

Then go deeper →

"Find any emails from my boss in the last 7 days and summarise them."

"Which threads have the word "urgent" in them? List them."

"Fetch the full content of the most recent email from [a colleague's name]."

Troubleshooting Guide

The 4 most common issues — and exact fixes.

✗ Access blocked: app not verified

✓ Go back to OAuth Consent Screen → Add Test Users → add your own Gmail address → try again.

✗ Cannot find module / node not recognized

✓ Node.js isn't installed or terminal wasn't restarted after install. Run: `node --version`. If error, reinstall Node LTS and reopen VS Code.

✗ credentials.json not found

✓ The CREDENTIALS_PATH in your Cline config doesn't match where the file actually is. Open File Explorer, find `credentials.json`, copy the full path and paste it in.

✗ Gmail server shows as disconnected in Cline

✓ Check your JSON config for syntax errors — missing comma, single backslash instead of double. Use jsonlint.com to validate. Then click "Restart MCP Servers" in Cline.

What You Did Today

Two days in. The agent is reading your real email.



01

Created a Google Cloud project and enabled the Gmail API



02

Generated OAuth credentials — the key to your inbox



03

Installed and configured the open-source Gmail MCP server



04

Completed the OAuth flow — token.json saved, never again



05

Agent listed, searched, and fetched real emails from your inbox

Tomorrow — MS 1.3

The Sorter

Feed the thread list to Gemini. Get back priority labels: URGENT / NEEDS-REPLY / FYI / IGNORE.

The triage function you'll use all month.

No homework tonight — just ship the post.



Post your inbox screenshot →
#MyAIChiefOfStaff

Sprint 1 progress:

MS 1.1 ✓

MS 1.2 ✓

MS 1.3

Sprint Build